Sistema de Gestión de Ventas

2025

Maximo Medina
Camila Tabone
Sebastian Sanabria

Universidad Atlántida Argentina
Programación III

Introducción

Este proyecto consiste en el desarrollo de una aplicación de escritorio destinada a la gestión de ventas, utilizando Java con Swing para la interfaz gráfica y Hibernate/JPA como framework ORM para interactuar con una base de datos relacional implementada en SQL Server. El entorno de desarrollo utilizado es Apache NetBeans IDE 23.
El sistema permite realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre entidades clave como Cliente, Vendedor, Producto, Estado, Territorio, Pedido y Detalle Pedido, manteniendo integridad relacional y usabilidad a través de una interfaz amigable.

Objetivos

- Diseñar un sistema de gestión de ventas funcional y modular.
- Integrar tecnologías como Hibernate, JPA y Java Swing.
- Mapear una base de datos relacional a objetos utilizando Hibernate.
- Permitir la gestión de pedidos y sus detalles desde una GUI.
  Utilizar hilos de ejecución (threads) para tareas de carga intensiva o en segundo plano sin afectar la experiencia del usuario.

Tecnologías Utilizadas

Tecnología	Propósito
Apache NetBeans IDE 23	Entorno de desarrollo
Java Swing	Interfaz gráfica de usuario
Hibernate / JPA	Framework ORM para persistencia
SQL Server (SSMS)	Base de datos relacional
Java Threads / ExecutorService	Procesamiento en segundo plano
PlantUML	Generación de diagramas UML
Git/Github	Control de versiones y colaboración remota

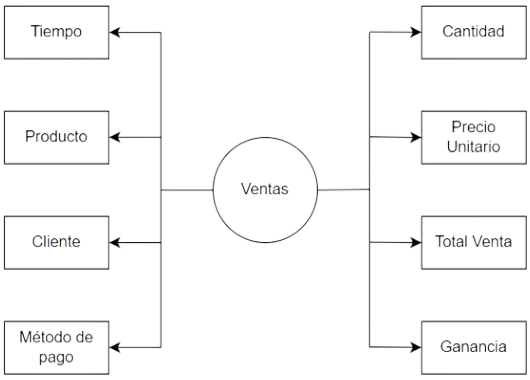
Modelo de Base de Datos

La base de datos contiene las siguientes entidades principales:
Cliente: Cliente ID (PK)
Producto: Producto ID (PK), producto, categoria, subcategoria, Precio Individual
Pedido: Pedido ID(PK), fechaPedido, Cliente ID(FK), Vendedor ID(FK), Territorio ID(FK), Estado ID(FK), SubTotal, Impuestos
Detalle del Pedido: Detalle ID(PK), Pedido ID(FK), Producto ID(FK), Cantidad, Precio Total, costo Envío
Vendedor: Vendedor ID(PK)
Estado: Estado ID(PK), estado
Territorio: Territorio ID(PK), Territorio

Relaciones:

- Un cliente puede hacer muchos pedidos.
- Un pedido tiene muchos detalles de pedido, cada uno vinculado a un producto.
- Cada detalle de pedido pertenece a un único producto.
- Un pedido puede estar vinculado a un único vendedor, un único territorio, un único estado, y un único método de envío.

Modelo Conceptual

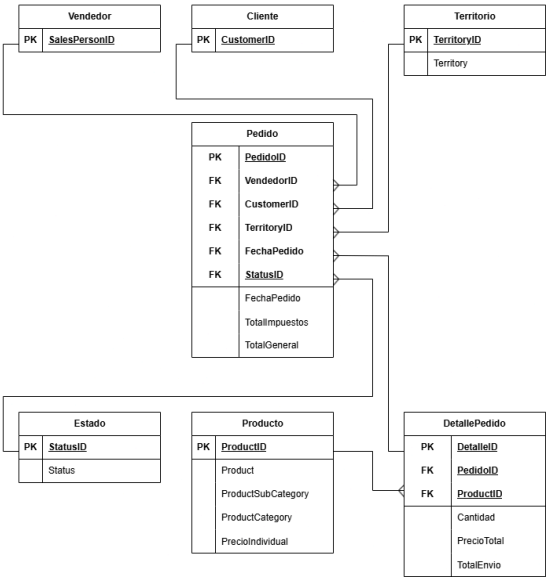


Este diagrama conceptual representa las principales entidades que intervienen en el proceso de una venta:
Ventas se vincula con los conceptos de:

- Tiempo (fecha o período)
- Producto vendido
- Cliente que realiza la compra
- Método de pago utilizado
- Además, contiene atributos como:
    - Cantidad, Precio Unitario, Total de Venta y Ganancia.

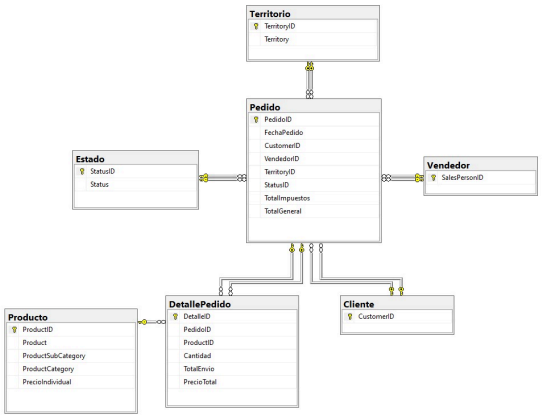
Este esquema ayuda a visualizar desde una perspectiva de negocio los componentes clave que posteriormente serán modelados en la base de datos.

Modelo Lógico



Este es el modelo relacional que se utilizó como base para generar las entidades Java con Hibernate. Muestra:
Las tablas principales: Pedido, DetallePedido, Cliente, Producto, Vendedor, Estado, Territorio.
Las relaciones entre tablas, representadas por claves foráneas (FK):
Pedido está relacionado con:
Cliente (quién hace el pedido)
Vendedor (quién lo gestiona)
Territorio (zona asignada)
Estado (estado del pedido)
DetallePedido depende de Pedido y Producto.
Este modelo permitió generar las clases anotadas con JPA (@Entity, @ManyToOne, @OneToMany) con integridad referencial automática.

Modelo Físico

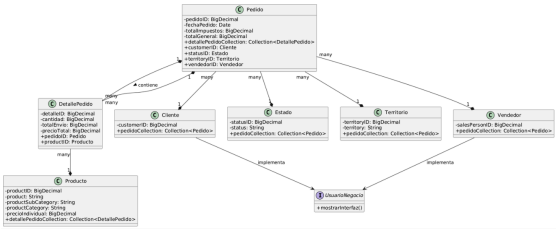


Este diagrama fue generado directamente en SQL Server Management Studio (SSMS) y representa la implementación física de las tablas en la base de datos.
Se puede observar:

- La configuración de claves primarias (PK) y foráneas (FK).
- Las relaciones uno a muchos entre:
    - Pedido y DetallePedido
    - Cliente y Pedido
    - Producto y DetallePedido
- La estructura completa que sirvió como base para generar las entidades persistentes en el proyecto Java.

Este modelo fue exportado y utilizado por Hibernate para crear las clases de persistencia mediante ingeniería inversa o mapeo manual.

Diagrama UML



- Pedido es la entidad central del modelo, relacionada con:
    - Cliente, quien realiza el pedido.
    - DetallePedido, que contiene uno o más productos por cada pedido.
    - Estado, que indica el estado del pedido (por ejemplo: En proceso, Enviado, Cancelado).
    - Territorio, que representa la zona geográfica asignada.
    - Vendedor, quien gestiona o genera el pedido.
- DetallePedido representa cada ítem individual de un pedido y se vincula con un Producto específico.
- Producto contiene los datos del artículo vendido, como categoría, subcategoría, nombre y precio individual.
- Las relaciones entre clases están expresadas mediante asociaciones:
    - 1 a muchos (ej. un cliente puede tener muchos pedidos).
    - muchos a 1 (ej. muchos detalles pertenecen a un mismo producto).
- Además, se incorporó el concepto de roles de usuario, modelados con la interfaz UsuarioNegocio, que es implementada por las clases Cliente y Vendedor. Esto permite que ambos tipos de usuario compartan una funcionalidad común, como la visualización de su propia interfaz en la aplicación (mostrarInterfaz()), respetando el principio de programación orientada a interfaces.

Ventajas del modelo:

- Claridad estructural: Las relaciones de cardinalidad y herencia están bien definidas, permitiendo una implementación directa y coherente en la base de datos.
- Escalabilidad: La arquitectura orientada a objetos permite agregar fácilmente nuevas entidades o relaciones.
- Reutilización de código: La interfaz común entre Cliente y Vendedor permite compartir lógica de presentación o negocio entre distintos tipos de usuarios.

Este diagrama es una referencia visual clave para comprender la organización del código, las dependencias entre entidades, y sirve como puente entre el diseño lógico de la base de datos y la implementación en Java.

Arquitectura de la Aplicación

Se utilizó Hibernate/JPA para mapear las tablas a clases Java.A continuación un ejemplo del mapeo de la clase entidad Pedido:

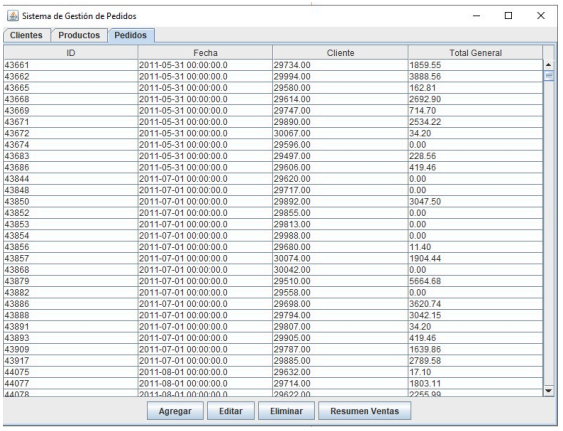
```java
public class Pedido implements Serializable {

    private static final long serialVersionUID = 1L;
    // @Max(value=?)  @Min(value=?)//
    @Id
    @Basic(optional = false)
    @Column(name = "PedidoID")
    private BigDecimal pedidoID;
    @Basic(optional = false)
    @Column(name = "FechaPedido")
    @Temporal(TemporalType.TIMESTAMP)
    private Date fechaPedido;
    @Basic(optional = false)
    @Column(name = "TotalImpuestos")
    private BigDecimal totalImpuestos;
    @Basic(optional = false)
    @Column(name = "TotalGeneral")
    private BigDecimal totalGeneral;
    @OneToMany(cascade = CascadeType.ALL, mappedBy = "pedidoID")
    private Collection<DetallePedido> detallePedidoCollection;
    @JoinColumn(name = "CustomerID", referencedColumnName = "CustomerID")
    @ManyToOne(optional = false)
    private Cliente customerID;
    @JoinColumn(name = "StatusID", referencedColumnName = "StatusID")
    @ManyToOne(optional = false)
    private Estado statusID;
    @JoinColumn(name = "TerritoryID", referencedColumnName = "TerritoryID")
    @ManyToOne(optional = false)
    private Territorio territoryID;
    @JoinColumn(name = "VendedorID", referencedColumnName = "SalesPersonID")
    @ManyToOne(optional = false)
    private Vendedor vendedorID;
```

Cada entidad tiene su clase controladora que permite hacer operaciones CRUD:

```java
//_____________PEDIDO_____________
public void crearPedido(Pedido pedido){
    controlP.crearPedido(pedido);
}
public void eliminarPedido(int id){
    controlP.eliminarPedido(id);
}
public void  editarPedido(Pedido pedido){
    controlP.editarPedido(pedido);
}
public Pedido traerPedido(int id){
    return controlP.traerPedido(id);
}
public List<Pedido> traerPedidos(){
    return controlP.traerPedidos();
}
```

Se utilizó java Swing para crear ventanas como:



ID	Fecha	Cliente	Total General
43661	2011-05-31 00:00:00.0	29734.00	1859.55
43662	2011-05-31 00:00:00.0	29994.00	3888.56
43665	2011-05-31 00:00:00.0	29580.00	162.81
43668	2011-05-31 00:00:00.0	29614.00	2692.90
43669	2011-05-31 00:00:00.0	29747.00	714.70
43671	2011-05-31 00:00:00.0	29890.00	2534.22
43672	2011-05-31 00:00:00.0	30067.00	34.20
43674	2011-05-31 00:00:00.0	29596.00	0.00
43683	2011-05-31 00:00:00.0	29497.00	228.56
43686	2011-05-31 00:00:00.0	29606.00	419.46
43844	2011-07-01 00:00:00.0	29620.00	0.00
43848	2011-07-01 00:00:00.0	29717.00	0.00
43850	2011-07-01 00:00:00.0	29892.00	3047.50
43852	2011-07-01 00:00:00.0	29855.00	0.00
43853	2011-07-01 00:00:00.0	29813.00	0.00
43854	2011-07-01 00:00:00.0	29988.00	0.00
43856	2011-07-01 00:00:00.0	29680.00	11.40
43857	2011-07-01 00:00:00.0	30074.00	1904.44
43868	2011-07-01 00:00:00.0	30042.00	0.00
43879	2011-07-01 00:00:00.0	29510.00	5664.68
43882	2011-07-01 00:00:00.0	29558.00	0.00
43886	2011-07-01 00:00:00.0	29698.00	3620.74
43888	2011-07-01 00:00:00.0	29794.00	3042.15
43891	2011-07-01 00:00:00.0	29807.00	34.20
43893	2011-07-01 00:00:00.0	29905.00	419.46
43909	2011-07-01 00:00:00.0	29787.00	1539.86
43917	2011-07-01 00:00:00.0	29885.00	2789.58
44075	2011-08-01 00:00:00.0	29632.00	17.10
44077	2011-08-01 00:00:00.0	29714.00	1803.11
44078	2011-08-01 00:00:00.0	29622.00	2255.99

Diseño basado en roles con la interfaz UsuarioNegocio

Para gestionar la presentación y comportamiento diferenciado según el rol de usuario, se definió la interfaz UsuarioNegocio, que establece un contrato común para los diferentes tipos de usuarios del sistema, principalmente los roles de Cliente y Vendedor.
Esta interfaz contiene un método fundamental:

```java
public interface UsuarioNegocio {
    void mostrarInterfaz();
}
```

El propósito de este método es que cada tipo de usuario implemente su propia versión de la interfaz gráfica, mostrando un dashboard o ventana específica adaptada a sus necesidades. Las clases Cliente y Vendedor implementan la interfaz UsuarioNegocio, definiendo ventanas distintas con Java Swing para cada rol. Esto permite presentar experiencias diferenciadas y optimizadas según el perfil de usuario.

Conclusiones

Este proyecto demostró la integración efectiva de diversas tecnologías en una aplicación de escritorio funcional. Algunos puntos destacados:

- Hibernate permitió desacoplar la lógica de negocio de la base de datos, facilitando el mantenimiento.
- La implementación de hilos mejoró la experiencia del usuario evitando bloqueos en la interfaz.
- Java Swing, aunque clásico, sigue siendo útil para aplicaciones rápidas y personalizadas.
- La estructura modular y el uso de JPA hicieron que el sistema sea escalable y extensible.

Mejoras futuras

- Implementar autenticación de usuarios (login).
- Exportar pedidos a PDF o Excel.
- Agregar búsqueda avanzada y filtros por cliente, fecha o producto.
- Reemplazar Swing por JavaFX para interfaces más modernas.
- Agregar la edición de la base de datos.
- Aplicación de la interfaz “UsuarioNegocio” para la visualización separada.

Anexo

Repositorio del proyecto:
https://github.com/MximPrtot/Tp-Progra